iLearn Practice Exam

10BR · Chapter / Topic 8

Cambridge IGCSE Computer Science 0478

Paper 2 Algorithms, Programming and Logic

Chapter 8: Programming

Name: _____

Class: _____

Date: _____

Time allowed: 1 hour 30 minutes

TOTAL MARKS: 75

Marks obtained: ..... / 75

INSTRUCTIONS

· Answer all questions.
· Write your answers in the spaces provided on this paper, using a black or dark blue pen.
· Do not use an erasable pen or correction fluid.
· No calculators are allowed.
· Do not write anything in the margins.

INFORMATION

· The total mark for this paper is 75.
· The number of marks for each question or part question is shown in brackets [ ].
· This paper contains 7 questions, all drawn from Cambridge past papers on this chapter.
· After you finish, your teacher will mark your work against the official mark scheme.

Questions reproduced from Cambridge International past papers © Cambridge University Press & Assessment, for classroom practice.

1 State **three** different features of a high-level programming language that a programmer could use to make sure that their program will be easier to understand by another programmer.
Give an example for each feature.

Feature 1 .....

.....

Example .....

.....

Feature 2 .....

.....

Example .....

.....

Feature 3 .....

.....

Example .....

.....

[6]

MASTER · iLEARN CS · LEARN · PRACTISE · iLearn CS

2 A weather station takes temperature readings once an hour for a week. These temperatures are stored in a two-dimensional (2D) array `Temperatures[]`
Each column contains 24 readings for a single day. The first temperature is recorded at 00:00 and the final temperature at 23:00. There are seven columns, one for each day of the week, starting with Monday and ending with Sunday.

The variables `MaxDay`, `MinDay` and `AvDay` are used to store the maximum, minimum, and average temperatures for a day. The variables `MaxWeek`, `MinWeek` and `AvWeek` are used to store the maximum, minimum, and average temperatures for the week.

The array has already been set up and the data stored.

Write a program that meets the following requirements:

- finds the maximum and minimum temperatures for each day
- calculates the average temperature for each day
- outputs for each day:
  - name of the day, for example Monday
  - maximum temperature
  - minimum temperature
  - average temperature
- finds the maximum and minimum temperatures for the week
- calculates the average temperature for the week
- outputs:
  - maximum temperature for the week
  - minimum temperature for the week
  - average temperature for the week.

All temperatures output must be rounded to two decimal places.

You must use pseudocode or program code **and** add comments to explain how your code works. All inputs and outputs must contain suitable messages.

You do **not** need to declare any arrays or variables; you may assume that this has already been done.
You do **not** need to initialise the data in the array `Temperatures[]`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

3 Describe **two** types of iteration that a programmer can use whilst writing a program.

.....

.....

.....

.....

.....

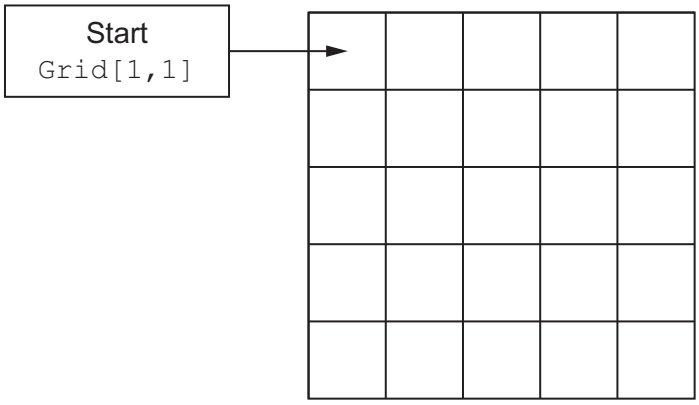
.....

.....

..... [4]

MASTER · iLEARN CS · LEARN · PRACTISE · iLearn CS

4 A one-player game uses the two-dimensional (2D) array `Grid[]` to store the location of a secret cell to be found by the player in 10 moves. Each row and column has 5 cells.



At the start of the game:

- The program places an 'X' in a random cell (**not** in `Grid[1,1]`) and empties all the other cells in the grid.
- The player starts at the top left of the grid.
- The player has 10 moves.

During the game:

- The player can move left, right, up or down by one cell and the move must be within the grid.
- "You Win" is displayed if the player moves to the cell with 'X' and has played 10 moves or less.
- "You Lose" is displayed if the player has made 10 moves without finding the 'X'.

Write a program that meets these requirements.

You must use pseudocode or program code **and** add comments to explain how your code works.

You do **not** need to declare any arrays or variables; you may assume that this has already been done.

All inputs and outputs must contain suitable messages.

.....

.....

.....

.....

.....

.....

.....

.....

BLANK PAGE

5 An incomplete algorithm has been written in pseudocode to count the number of zeros stored in an array and total the non-zero values.

```
01 DECLARE A[1:50] : INTEGER
02 DECLARE C : INTEGER
03 DECLARE I : INTEGER
04 DECLARE T : INTEGER
05 I ← 0
06 .....
07 FOR C ← 1 TO 50
08     IF A[C] .....
09       THEN
10         T ← T + 1
11       ELSE
12         I ← I + A[C]
13     ENDIF
14 .....
```

(a) Complete the given pseudocode algorithm. [3]

(b) Write the pseudocode to display, with suitable messages, the number of zeros stored in the array and the total of the non-zero values.

.....

.....

.....

.....

.....

..... [3]

(c) Meaningful identifiers have **not** been used in the algorithm.
Suggest suitable meaningful identifiers for:

The array:

A .....

The variables:

T .....

C .....

I .....

6 This pseudocode algorithm should allow 500 names of people and their heights to be entered and stored in two one-dimensional (1D) arrays. The name of a person and the height of that person are in the same index in both the `Names[]` and the `Heights[]` arrays.

The algorithm performs as follows:

- at the start, the following data is entered:
  - name of person
  - height of person in cm
- the data is stored in the relevant arrays
- the heights of the people are totalled
- the shortest person is found
- at the end, the following is output:
  - name of shortest person
  - height of shortest person
  - average height of all 500 people.

```
01  DECLARE Names : ARRAY[1:500] OF REAL
02  DECLARE Heights : ARRAY[1:500] OF REAL
03  DECLARE Shortest : REAL
04  DECLARE Total : REAL
05  DECLARE Counter : INTEGER
06  DECLARE Index : INTEGER
07  Shortest ← 500
08  Index ← 0
09  Total ← 100
10  FOR Counter ← 1 TO 500
11      INPUT Names[Index]
12      INPUT Heights[Counter]
13      Total ← Counter + Heights[Counter]
14      IF Heights[Counter] < Shortest
15        THEN
16          Shortest ← Heights[Counter]
17          Index ← Counter
18      ENDIF
19  NEXT Counter
20  OUTPUT "The shortest height is ", Heights
21  OUTPUT "The shortest person is ", Names[Index]
22  OUTPUT "The average height is ", Total / 500
```

(a) Identify the line numbers of **five** errors in the pseudocode and suggest a correction for each error.

Error 1 line number .....

Correction .....

.....

Error 2 line number .....

Correction .....

.....

Error 3 line number .....

Correction .....

.....

Error 4 line number .....

Correction .....

.....

Error 5 line number .....

Correction .....

..... [5]

(b) Write the pseudocode statement that will output the average height rounded to **one** decimal place.

.....

..... [2]

(c) Explain how you could change the corrected algorithm so that it will also:

- find the height of the tallest person.
- output the height of the tallest person at the end of the algorithm.

Any program statements used must be fully explained.

.....

.....

.....

.....

.....

.....

.....

7 The scores for each competitor in an archery competition are recorded.

The one-dimensional (1D) array `CompetitorName[]` is used to store the names of the competitors.

The maximum number of competitors is 30.

There are 10 rounds in the competition with a maximum score of 30 per round.

The two-dimensional (2D) array `CompetitorScore[]` stores the score in each of the 10 rounds for each competitor.

The first index of the array `CompetitorScore[]` is the same as the index of the competitor in `CompetitorName[]`

After the 10 rounds have been completed, the highest score and the lowest score for each competitor are discarded.

The overall score for each competitor is the total of the remaining 8 scores (after discarding the highest and lowest scores) .

A competitor with an overall score of 210 or more moves onto the next competition. A competitor with an overall score between 180 and 209 inclusive becomes a reserve competitor. A competitor with an overall score below 180 does not qualify.

Write a program that meets the following requirements:

- inputs the total score (out of 30) for each competitor in each round
- validates each input is between 0 and 30 inclusive
- calculates the overall score for each competitor discarding the highest and lowest scores
- outputs the name of each competitor and if they move onto the next competition, are a reserve or do not qualify
- calculates and outputs the number of competitors who:
  - have qualified for the next competition
  - are reserve competitors
  - do **not** qualify.

You must use pseudocode or program code **and** add comments to explain how your code works.

You do **not** need to declare any arrays or variables; assume that this has already been done.

All inputs and outputs must contain suitable messages.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [15]

MASTER · iLEARN CS · LEARN · PRACTISE
iLearn CS

BLANK PAGE

MASTER · iLEARN CS · LEARN · PRACTISE · iLearn CS

END OF QUESTION PAPER